

GENERATIVE AI | CAPSTONE PROJECT 2026

VeriDoc

A PDF question-answering system that verifies its own answers

NAME	Anuruddha Paul
ROLL NUMBER	2328072
BATCH	GENAI 2026
SUBMITTED TO	Mr Sachin
DATE	30 September 2026

Project links

Source code: <https://github.com/AnuruddhaPaul/VeriDoc>

Live app: [to be added before submission]

Project folder (Google Drive): [to be added before submission]

1. Problem Statement

Many people now ask a chatbot questions about a document instead of reading it: an employee handbook, a rental contract, a company policy. Most of these tools use retrieval-augmented generation (RAG): they find the passages closest to the question and let a language model write an answer from them. The weak point is what happens when the document does **not** contain the answer. A model that is eager to help can fill the gap with a believable guess, and the reader has no way to tell a sentence that came from page 3 from one that the model made up. For a leave policy or a contract clause, a wrong answer that sounds sure of itself can cost real money or time.

VeriDoc attacks that specific problem. It does not just answer; it checks each claim in its own answer against the retrieved text before showing it, and it says “Not found in the document” when the document does not back the claim.

Domain	Question answering over user-supplied PDF documents (demonstrated on an employee handbook)
Users	People who rely on PDF chatbots for handbooks, policies and contracts and need to trust the answer
Problem	RAG chatbots can answer confidently even when retrieval missed, and readers cannot separate grounded statements from invented ones
Success criteria	At least 90% of answerable questions answered correctly; at least 90% of unanswerable questions refused; every claim shown as supported carries a page number and a quote that can be found word for word in the document

A number that can be tested. Of the 40 questions used in this project (22 development + 20 held-out, of which 20 have no answer in the document), the target is that 18 or more of the 20 unanswerable questions get a refusal instead of an answer. Section 9 reports what was actually measured.

2. Objectives and Scope

The objectives are written so that each one can be checked with a number. Targets were set before the final runs and are reported against in Section 13.

#	Objective (measurable)	How it is measured
O1	Answer at least 90% of answerable questions correctly	Answer rate on the two question sets (Section 9)
O2	Refuse at least 90% of unanswerable questions	Abstain rate: no answer shown
O3	Every answer marked “grounded” is correct (grounded precision of at least 95%)	Share of “grounded” answers that are correct
O4	A verifier failure is never shown as “grounded” (fail closed)	Unit tests for bad JSON and crashes
O5	Offer a verifier that runs locally, with no second API call, and compare it with the LLM verifier	Ablation: answer rate, latency, API calls
O6	Keep the project reproducible: at least 20 automated tests that run offline, and a public app	pytest result; hosted link

In scope	Out of scope
• Text PDFs (one or several) • Single-question answers with citations • Per-claim verification and evidence display • Comparison of LLM and NLI verifiers • Baseline mode with checking switched off	• Scanned PDFs that need OCR • Multi-turn conversation memory • Deciding whether the document itself is true • Tables, charts and images inside PDFs • Documents in languages other than English

3. Solution and Features

VeriDoc is a web page where a person uploads one or more PDFs, asks questions, and gets an answer together with proof of where each part came from. What the user can do:

- **Upload and ask.** Drop in a PDF (or load the built-in sample handbook) and type a question.
- **See a trust badge.** Each answer is labelled Grounded, Partly grounded, Not grounded, Not found, or Unverified, in a matching colour.
- **Read a claim-by-claim check.** The answer is split into single facts. Each fact gets a tick or a cross; a tick shows the source number and the page.
- **See the evidence.** The retrieved passages are listed with page and similarity, and the exact sentence that supports a claim is highlighted.
- **Get a straight refusal.** If the document does not contain the answer, the app says so, and for clearly off-topic questions it does not even call the language model.
- **Choose the checker.** Switch between the LLM verifier, the local NLI verifier, or Off (plain RAG) to compare them, and optionally swap in a “typical chatbot” prompt for the comparison.
- **Tune it.** Sliders for the number of retrieved passages and for the off-topic cutoff.

4. Tech Stack

Technology	Purpose in this project	Why it was chosen
PyMuPDF	Extracts text from each PDF page	Fast, pure pip install, gives text blocks per page so every chunk keeps an exact page number
Sentence-Transformers (all-MiniLM-L6-v2)	Turns chunks and questions into 384-number vectors	Small (about 90 MB), runs on CPU, free, and no text leaves the machine for embedding
ChromaDB	Stores vectors and finds the nearest chunks (cosine)	Runs inside the Python process, so there is no database server to host or pay for
Groq API, openai/gpt-oss-120b	Writes the answer from the sources and, in one mode, checks it	Free tier and very fast responses. It replaced Llama-3.3-70B, which Groq had retired (Section 11)
cross-encoder/nli-deberta-v3-small	Local entailment model for the NLI verifier	Small enough for CPU, needs no API call, and gives a numeric score for each claim
Streamlit	The web interface and the server in one	A full UI in plain Python; no separate front end or back end to build
pytest	25 offline tests, using a scripted fake LLM and a hashing embedder	Tests run in seconds with no internet or API key, so they can be run on every change
Playwright (Chromium)	Drives the real app to capture the screenshots	Scripted, repeatable screenshots at a fixed window size instead of manual captures
Streamlit Community Cloud	Public hosting of the app	Free, deploys straight from a GitHub repository, and stores the API key as a secret

5. Knowledge Base and Data

The document

To test refusal honestly, the document must contain facts that no language model could know. So the test document is a fictional “Northwind Robotics Employee Handbook”, created for this project with a short script (`sample_docs/make_sample_pdf.py`) using PyMuPDF. It has **5 pages**, one topic per page: leave, remote work, travel and expenses, information security, and benefits and learning. Every number in it (25 days of annual leave, a 600 USD stipend, 14-character passwords and so on) is invented. If the system states a fact that is not in these five pages, it can only have come from the model’s imagination.

Chunking and retrieval

Setting	Value	Reason
Chunk size	about 350 tokens	Large enough to hold a whole policy section, small enough to be specific
Overlap	about 60 tokens	A fact sitting on a chunk boundary is not lost
Boundaries	sentence-aware; never across pages	Every citation gets an exact page number
Embedding	all-MiniLM-L6-v2, normalised	Cosine similarity equals a dot product
Chunks produced	5 (one per page)	Each page is shorter than 350 tokens, so a page is one chunk
top-k	4	The best four chunks go to the generator
Off-topic cutoff	0.15 (cosine)	If even the best chunk scores below this, no LLM call is made

Honest limitation. With only 5 chunks and top-k = 4, retrieval sends about 80% of the whole document to the model on every question. Retrieval is therefore easy in this test, and the results say little about how the system copes with a 200-page PDF. That is a limit of the evaluation, not of the design.

The question sets

Set	Answerable	Unanswerable	Notes
Development (<code>eval/questions.json</code>)	12	10 (8 in-domain but absent, 2 off-topic)	Used while building and diagnosing
Held-out (<code>eval/questions_heldout.json</code>)	10 (paraphrased)	10 (6 near-miss, 4 in-domain absent)	Written before the final runs; run once with settings frozen

A “near-miss” question sits right next to something the handbook does say, for example the international meal limit when only the domestic limit is given. These are the questions where a careless model is most tempted to guess.

6. System Architecture

Figure 6.1 shows the whole system. Ingestion happens once per PDF; the query lane runs for every question. The two diamonds are the two places where the system can stop early and refuse; the red boxes are those exits.

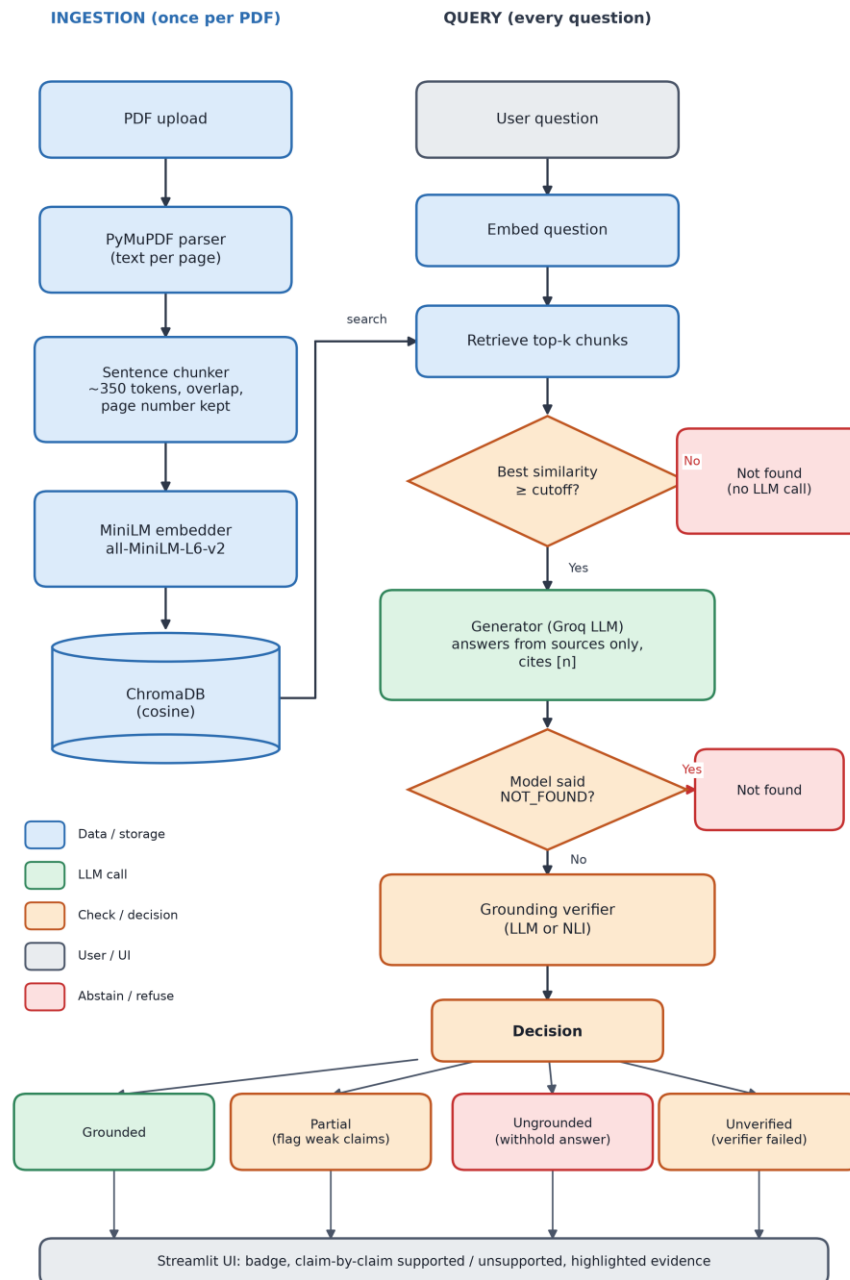


Figure 6.1: System architecture. Blue = data and storage, green = LLM call, orange = check or decision, grey = user or UI, red = refusal.

One request from start to finish. Take the question “How many days of annual leave do full-time employees get?” (a real run from the live tests). The question is embedded with the same MiniLM model and the four nearest chunks are fetched; the best has similarity 0.61, well above the 0.15 cutoff. The generator receives only those chunks and replies “Full-time employees receive 25 days of

paid annual leave per calendar year [1].” The verifier splits that into one claim, cites source 1 and quotes “All full-time employees receive 25 days of paid annual leave per calen...”. Code confirms the quote really is in chunk 1, so the claim stays supported, all claims are supported, and the UI shows a green Grounded badge with page 1.

Now the question “What is the capital of France?”. The best similarity is -0.03 , below the cutoff, so the system answers “Not found in the document” without calling the model at all.

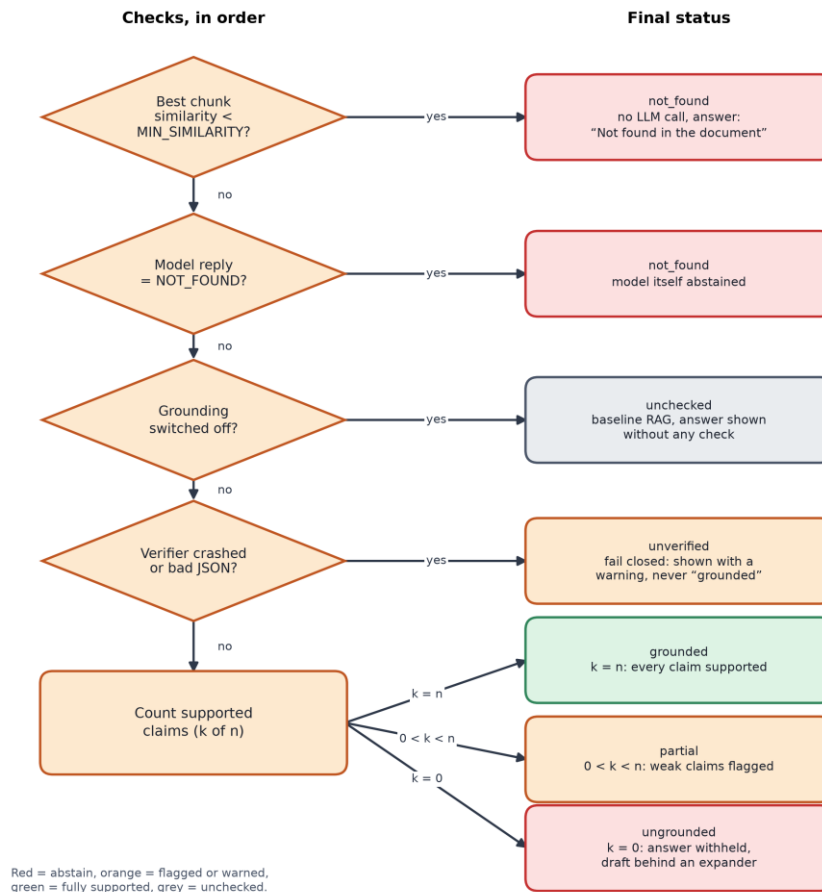


Figure 6.2: Decision logic in `pipeline.py`. The checks run top to bottom; the first one that fires decides the status. k is the number of supported claims and n the number of claims.

The order matters. The cheap similarity check comes first so off-topic questions cost nothing. Verifier failure comes before the count of supported claims so that a broken checker can never produce a “grounded” label. This is the fail-closed rule (objective O4).

7. Implementation Highlights

7.1 The verifier cannot simply assert support

A second language model checking the first is only as trustworthy as its own honesty. So the LLM verifier is not allowed to just say “supported”. Each supported claim must name a source and give a short quote, and ordinary code then checks that the quote appears in that source (ignoring case and punctuation). An invented quote or a wrong source number turns the claim into unsupported.

Excerpt from veridoc/grounding.py (abridged)

```
def quote_in_text(quote: str, text: str) -> bool:
    """True if every '...' -separated segment of `quote` appears verbatim in `text`."""
    haystack = _normalise(text)
    segments = [_normalise(s) for s in re.split(r"\.{3}|\u2026", quote)]
    segments = [s for s in segments if s]
    return bool(segments) and all(s in haystack for s in segments)

# inside LLMVerifier.verify():
if supported and (rank is None or not quote_in_text(quote, retrieved[rank - 1].chunk.text)):
    supported = False # claimed support without checkable evidence
```

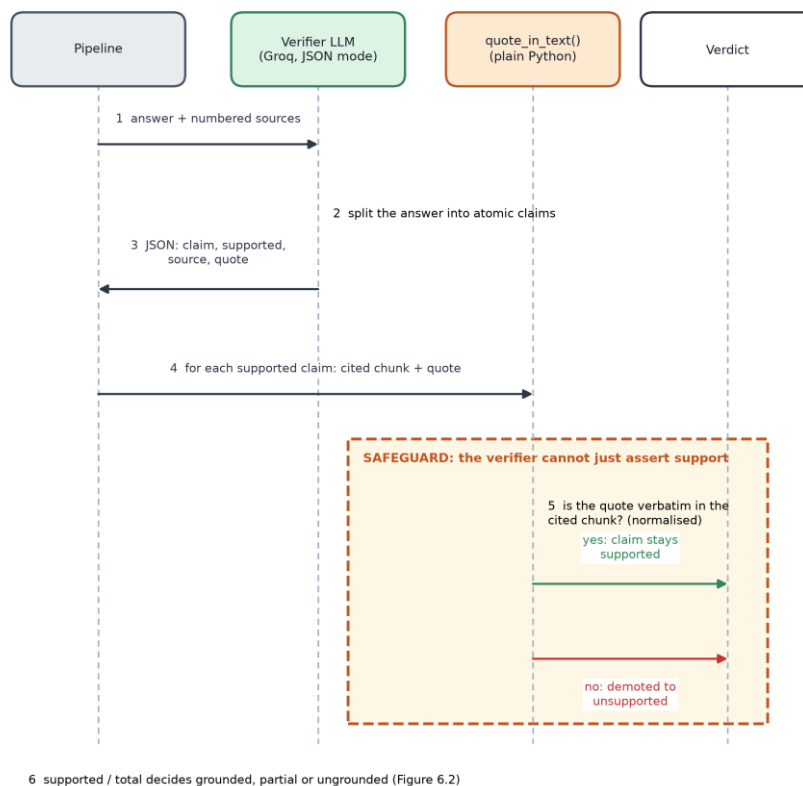


Figure 7.1: LLM verifier sequence. The dashed orange box is the safeguard: the quote check is plain code, not another model opinion.

7.2 The similarity gate and the fail-closed decision

Excerpt from `veridoc/pipeline.py` (abridged)

```
if not retrieved or retrieved[0].score < self.min_similarity:
    return Result(question, NOT_FOUND, NOT_FOUND_MESSAGE, ...) # no LLM call at all
...
try:
    report = self.verifier.verify(draft, retrieved)
except VerificationError:
    return Result(question, UNVERIFIED, draft, ...) # fail closed, never
    "grounded"
...
if report.supported_count == len(report.verdicts): status = GROUNDED
elif report.supported_count == 0: status = UNGROUNDED # answer withheld
else: status = PARTIAL
```

7.3 Scoring single sentences for the NLI verifier

The NLI model was first given only three-sentence windows of each chunk as evidence. In the live test the same claim scored 0.998 against the one sentence that states it, but 0.002 once a neighbouring sentence was added to the evidence. The fix is to offer every single sentence as well as the windows and keep the best score.

Excerpt from `veridoc/grounding.py`, `NLIVerifier._windows` (abridged)

```
out.extend((rank, s) for s in sentences) # every single sentence
if self.window > 1: # plus sliding multi-sentence
    windows
    for i in range(0, max(1, len(sentences) - self.window + 2), max(1, self.window - 1)):
        out.append((rank, " ".join(sentences[i : i + self.window])))
```

8. Testing and Results

Automated tests. 25 tests pass in about 4 seconds with no internet and no API key. They use a scripted fake language model and a deterministic hashing embedder, plus a headless run of the whole Streamlit script. The original suite had 21 tests; 4 were added during live testing, one for each real bug described in Section 11. The NLI test was confirmed to fail when its fix is removed. One earlier bug was caught by the original tests: the model's refusal NOT_FOUND was not recognised because the underscore was compared literally.

The table lists the checks that matter most, **including the ones that failed**.

Input	Expected	Actual	Status
Unit test: invented quote from the verifier	Claim demoted to unsupported	Demoted	Pass
Unit test: verifier returns bad JSON / crashes (3 cases)	Status unverified, never grounded	unverified	Pass
Unit test: best similarity below cutoff	Not found, zero LLM calls	Not found, 0 calls	Pass
Live: first Groq call with llama-3.3-70b-versatile	Model replies	HTTP 404 model_not_found (retired)	Fail , fixed by switching to gpt-oss-120b
Live: NLI, claim vs two-sentence evidence	High entailment (claim is stated)	0.002 (0.998 for the single sentence)	Fail , fixed with single-sentence evidence + test
Live: annual leave question, LLM verifier	Grounded, 25 days, quote from page 1	Grounded; quote verified	Pass
Live: "What is the bereavement leave policy?"	Not found	Not found (model said NOT_FOUND, similarity 0.43)	Pass

Input	Expected	Actual	Status
Live: “What is the capital of France?”	Not found, no LLM call	Not found; similarity –0.03	Pass
First full eval run (before output clean-up)	About 90%+ answered	NLI verifier answered 42%; baseline 83%	Fail: model output had odd spaces and 【 】 citations; scorer missed “three” vs 3. Both fixed
Baseline (grounding Off, typical prompt): “How many stock options do new hires receive?”	Guessed answer, to show the problem	Polite refusal: handbook does not mention stock options	Hypothesis not confirmed (no hallucination seen)
Typical prompt + LLM verifier on a correct refusal	Passes through as Not found	Marked Not grounded (5 of 8), Partial (2), Grounded (1)	Known weakness , not fixed
NLI: “...reported to the security team within 2 hours”	Supported	Entailment 0.068, rejected	Known weakness of the small NLI model
Groq free-tier daily token cap reached mid-project	App keeps working	HTTP 429; a run crashed	Fail , fixed: automatic fallback model + test

Live demo script. In the running app I asked: two answerable questions (annual leave with the LLM verifier; minimum password length with the NLI verifier); one unanswerable question with the normal settings (bereavement leave); the same bereavement question and a stock-options question with grounding off and the typical-chatbot prompt; the bereavement question again with the typical prompt plus the LLM verifier; and one off-topic question (capital of France). The screenshots in Section 10 are from these runs. A partly-grounded status did occur naturally in the typical-prompt evaluation (2 of 8 questions), but not in my manual demo, so there is no screenshot of it; a unit test covers it.

9. Evaluation and Metrics

Method

Each configuration was run over the same questions against the same document. **Answerable questions** are marked correct if the shown answer contains every expected fact (for example “25”), after normalising spaces and turning number words such as “three” into digits. **Unanswerable questions** are marked “answered” if any answer text is shown; a refusal, or an answer withheld by the verifier, counts as abstaining. All calls use temperature 0 and the same generator, openai/gpt-oss-120b with low reasoning effort. Only the grounding check differs between rows.

Metric	Meaning
Answer rate	Answerable questions where a correct answer was shown (higher is better)
Abstain rate	Unanswerable questions where no answer was shown (higher is better)
Hallucination rate	Unanswerable questions where an answer was shown (lower is better)
Grounded precision	Of answers labelled fully grounded, the share that were correct

Results

Development set (12 answerable, 10 unanswerable), final run:

Configuration	Answer rate	Abstain rate	Hallucination rate	Grounded precision	Time (s)
Baseline RAG (no check)	100% (12/12)	100%	0% (0/10)	n/a	58
LLM verifier	100% (12/12)	100%	0% (0/10)	100%	156

Configuration	Answer rate	Abstain rate	Hallucination rate	Grounded precision	Time (s)
NLI verifier	92% (11/12)	100%	0% (0/10)	100%	99

Held-out set (10 answerable, 10 unanswerable), run once with settings frozen:

Configuration	Answer rate	Abstain rate	Hallucination rate	Grounded precision	Time (s)
Baseline RAG (no check)	100% (10/10)	100%	0% (0/10)	n/a	66
LLM verifier	100% (10/10)	100%	0% (0/10)	100%	159
NLI verifier	80% (8/10)	100%	0% (0/10)	100%	86

First run on the development set (before the output-formatting and scorer fixes), kept for honesty:

Configuration	Answer rate	Abstain rate	Hallucination rate	Grounded precision	Time (s)
Baseline RAG (no check)	83% (10/12)	100%	0% (0/10)	n/a	73
LLM verifier	83% (10/12)	100%	0% (0/10)	83%	183
NLI verifier	42% (5/12)	100%	0% (0/10)	83%	95

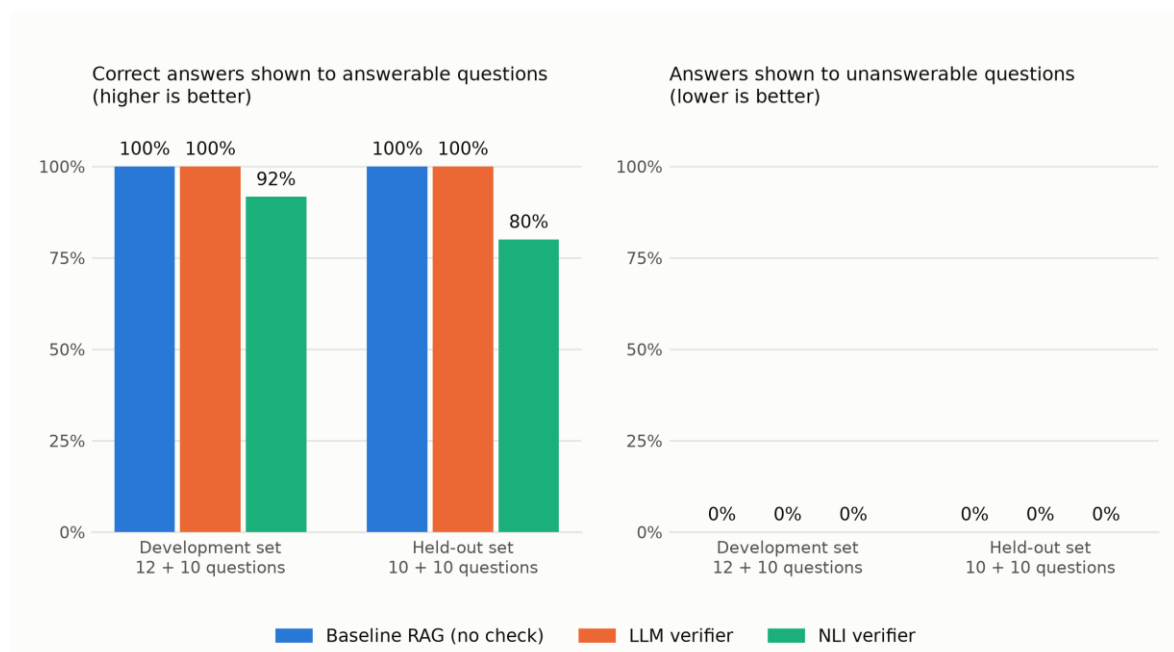


Figure 9.1: Answer rate (left) and hallucination rate (right) for the three configurations on both question sets. All values are read from the evaluation result files.

What the numbers say

- **Refusal worked on every configuration:** 0 of 20 unanswerable questions received a fabricated answer, on both sets, including 6 near-miss questions. With 0 events in 20 trials, the 95% upper bound on the true rate is still about 14%, so this is encouraging but not proof.
- **The LLM verifier kept the baseline's answer rate** (100% on the held-out set) and every answer it marked grounded was correct (grounded precision 100%).
- **The NLI verifier was stricter and lost answers:** 92% on the development set and 80% on the held-out set. It refused correct answers, so objective O1 is not met for the NLI verifier on held-out data. When I re-ran the three refused questions afterwards (on the fallback model, so the

wording of the answers differed), two were accepted, so some misses depend on how the answer is worded. One claim (“reported to the security team within 2 hours”, while the source says “laptop or phone”) scored only 0.013 and 0.068 in two separate checks.

- **The baseline did not hallucinate**, which does not support the assumption the project started from. With this generator, the strict prompt, and a five-page document, plain RAG already refused correctly.

Extra experiment: a typical chatbot prompt

To test whether the grounding layer helps when the generator is less careful, the strict prompt was replaced with a plain “use the context to answer” prompt, on the development set only (the held-out version of this run was stopped by the free-tier token cap).

Configuration	Answer rate	Unanswerable answered (automatic count)	Grounded precision
Typical prompt, no check	100%	80%	n/a
Typical prompt + LLM verifier	100%	30%	100%

The automatic count is misleading here, so I read all 10 answers. In the no-check row, all 8 in-domain answers were polite refusals such as “the handbook does not specify any stock-option grant”, and the 2 off-topic questions hit the similarity cutoff. **By hand, there were 0 real hallucinations out of 10.** The verifier lowered the automatic count from 80% to 30% only by withholding or flagging correct refusals (5 withheld, 2 partial, 1 marked grounded). This shows a real weakness: a refusal is a negative statement, and the verifier is built to find a quote that supports a positive claim.

Ablation: baseline vs LLM verifier vs NLI verifier

	Baseline	LLM verifier	NLI verifier
Extra API calls per answered question	0	1 (second Groq call)	0 (runs locally)
Time for the 22 development questions	58 s	156 s	99 s
Time per question (average)	2.6 s	7.1 s	4.5 s
Evidence shown	Sources only	Claim, source, verbatim quote	Claim, best sentence, score
Main risk	No check	Costs tokens; rejects refusals	Brittle on wording; loses answers

The NLI time includes loading the model once (about 6 s after the first download of about 40 s). The LLM verifier is the slowest because it is a full extra model call, but it explains itself with a quote. The NLI verifier is free to run and private, but in these tests it was the least reliable.

Tuning and limits

No thresholds were tuned. MIN_SIMILARITY (0.15), NLI_THRESHOLD (0.5) and top-k (4) are the original values. Between the first and final development runs I changed only things that were bugs: the output clean-up, the scorer, and the NLI evidence windows. The held-out set was written before those final runs and evaluated once. The limits are real: 22 + 20 questions is a small sample, one fictional 5-page document, one generator model, and the development questions were seen while diagnosing. The runs used one model; per-question model names were not recorded, and a fallback model was only added afterwards.

10. Screenshots

All screenshots are of the real running app at a 1400 px wide window, captured by a script (`docs/take_screenshots.py`). The sidebar API-key box is not shown because the key is loaded from the environment. The small grey line under some answers names the model; a few later screenshots show `gpt-oss-20b` because the free-tier daily token limit of the main model was reached and the app switched automatically.

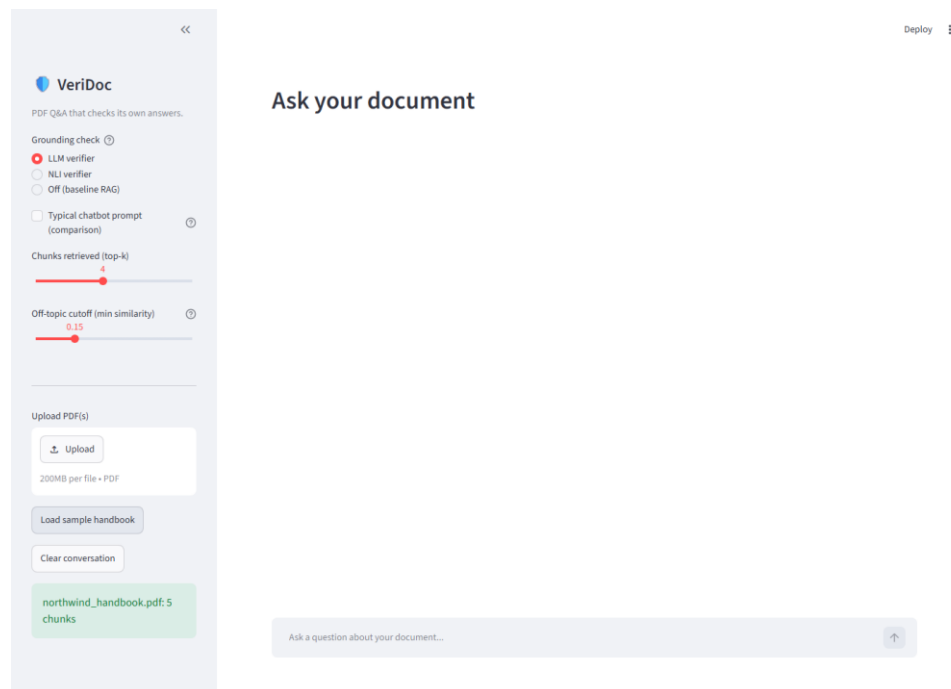


Figure 10.1: App home with the sample handbook loaded (5 chunks indexed) and the grounding controls in the sidebar.

VeriDoc
PDF Q&A that checks its own answers.

Grounding check ⓘ
☒ LLM verifier
☐ NLI verifier
☐ Off (baseline RAG)

☐ Typical chatbot prompt (comparison) ⓘ

Chunks retrieved (top-k)

Off-topic cutoff (min similarity) ⓘ

Upload PDF(s)

 200MB per file • PDF

northwind_handbook.pdf: 5 chunks

Ask your document

How many days of annual leave do full-time employees get?

Grounded - 1/1 claims supported

Full-time employees receive 25 days of paid annual leave per calendar year [1].

Claim-by-claim check (LLM)

✓ Full-time employees receive 25 days of paid annual leave per calendar year. — source [1], page 1

Retrieved sources (4)

[1] northwind_handbook.pdf · page 1 · similarity 0.61

1. Leave Policy All full-time employees receive 25 days of paid annual leave per calendar year. Unused leave may be carried over, up to a maximum of 5 days, and must be used by 31 March of the following year. Employees receive 10 days of paid sick leave per year. A medical certificate is required when sick leave exceeds 3 consecutive days. Parental leave is 16 weeks at full pay for the primary caregiver and 6 weeks at full pay for the secondary caregiver. Parental leave can be taken at any time within the first 12 months after the birth or adoption of a child. Northwind Robotics - Employee Handbook (fictional)

[2] northwind_handbook.pdf · page 2 · similarity 0.42

2. Remote Work Employees may work remotely up to 3 days per week, with the manager's agreement on which days. Everyone must be available during core hours, 10:00 to 15:00 in their local time zone. New employees must work on-site for the first 4 weeks while they are onboarded. The company pays a one-time home-office stipend of 600 USD after the probation period is complete. The stipend covers a desk, a chair or a monitor and cannot be used for other items. Northwind Robotics - Employee Handbook (fictional)

[3] northwind_handbook.pdf · page 5 · similarity 0.38

Ask a question about your document...

Figure 10.2: A grounded answer. The badge, the claim-by-claim tick with source and page, and the retrieved source with the supporting sentence highlighted in yellow.

VeriDoc
PDF Q&A that checks its own answers.

Grounding check ⓘ
☐ LLM verifier
☒ NLI verifier
☐ Off (baseline RAG)

☐ Typical chatbot prompt (comparison) ⓘ

Chunks retrieved (top-k)

Off-topic cutoff (min similarity) ⓘ

Upload PDF(s)

 200MB per file • PDF

northwind_handbook.pdf: 5 chunks

Ask your document

What is the minimum password length?

Grounded - 1/1 claims supported

Passwords must be at least 14 characters long [1].

Claim-by-claim check (NLI)

✓ Passwords must be at least 14 characters long. — source [1], page 4 · entailment 0.99

Retrieved sources (4)

[1] northwind_handbook.pdf · page 4 · similarity 0.57

4. Information Security Passwords must be at least 14 characters long and must be changed every 180 days. Multi-factor authentication is mandatory for all company systems. A lost or stolen laptop or phone must be reported to the security team within 2 hours of discovery. Employees must never share credentials, and confidential documents may only be stored on the company-approved cloud drive. Northwind Robotics - Employee Handbook (fictional)

[2] northwind_handbook.pdf · page 2 · similarity 0.19

2. Remote Work Employees may work remotely up to 3 days per week, with the manager's agreement on which days. Everyone must be available during core hours, 10:00 to 15:00 in their local time zone. New employees must work on-site for the first 4 weeks while they are onboarded. The company pays a one-time home-office stipend of 600 USD after the probation period is complete. The stipend covers a desk, a chair or a monitor and cannot be used for other items. Northwind Robotics - Employee Handbook (fictional)

[3] northwind_handbook.pdf · page 1 · similarity 0.17

1. Leave Policy All full-time employees receive 25 days of paid annual leave per calendar year. Unused leave may be carried over, up to a maximum of 5 days, and must be used by 31 March of the following year. Employees receive 10 days of paid sick leave per year. A medical certificate is required when sick leave exceeds 3 consecutive days. Parental leave is 16 weeks at full pay for the primary caregiver and 6 weeks at full pay for the secondary caregiver. Parental leave can be taken at any time within the first 12 months after the birth or adoption of a child. Northwind Robotics - Employee Handbook (fictional)

[4] northwind_handbook.pdf · page 3 · similarity 0.06

3. Travel and Expenses Domestic business travel meals are reimbursed up to 60 USD per day. Employees fly economy class for any flight shorter than 6 hours; premium economy is allowed for longer flights. Hotel stays are booked through the company travel portal. Expense reports must be submitted within 30 days of the expense. Reports submitted later than 30 days require written approval from a department head before they are reimbursed. Northwind Robotics - Employee Handbook (fictional)

retrieve 0.02s · generate 1.15s · verify 9.05s · model openai/gpt-oss-120b

Ask a question about your document...

Figure 10.3: The NLI verifier on “minimum password length”: the claim is supported with entailment 0.99, and the same evidence sentence is highlighted.

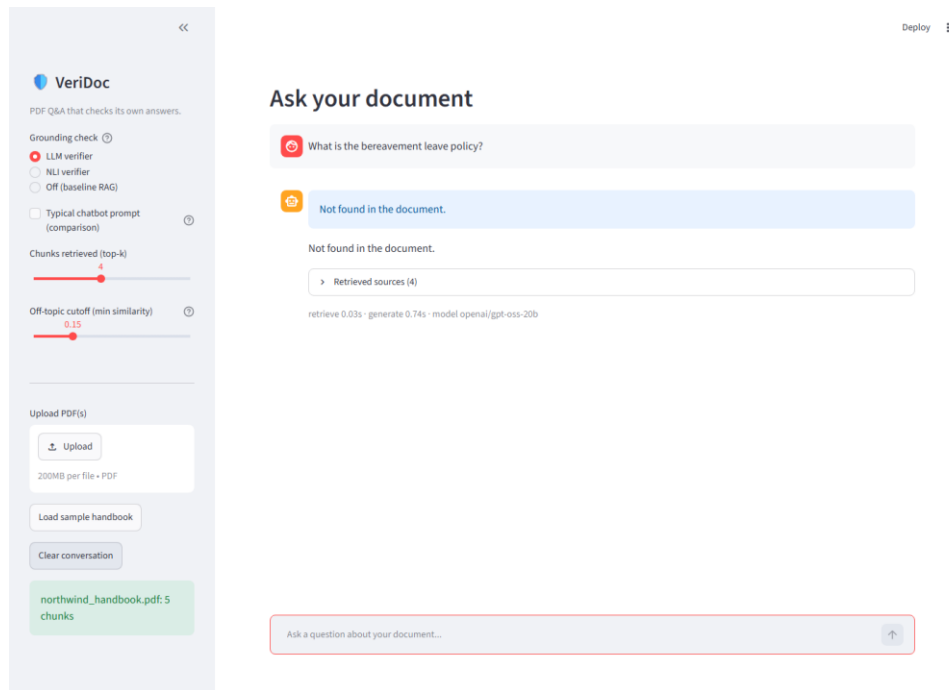


Figure 10.4: A question the document does not answer (bereavement leave). The model returned `NOT_FOUND` and VeriDoc shows “Not found in the document”.

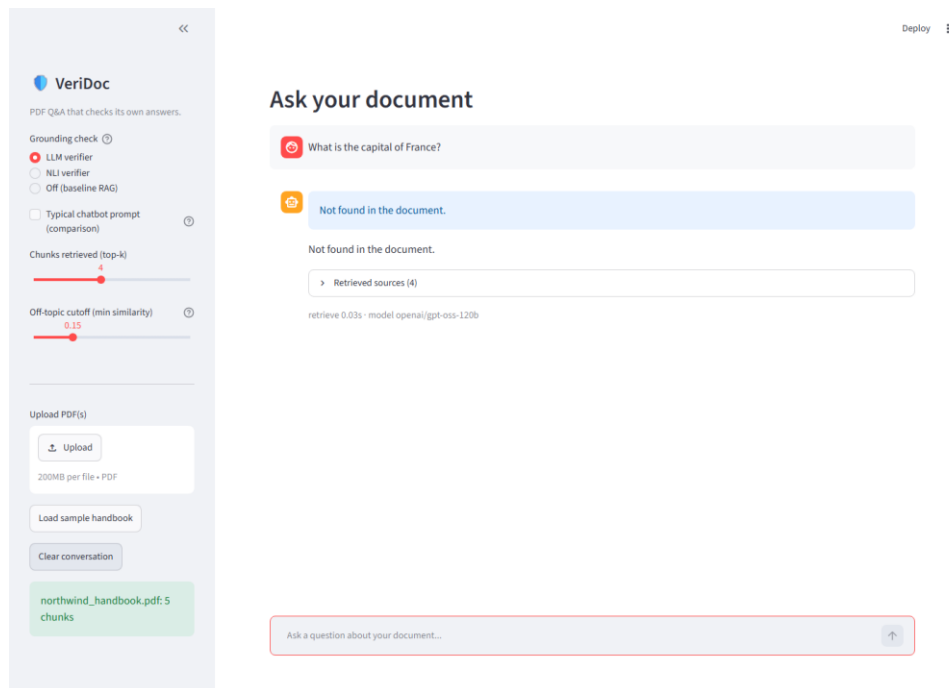


Figure 10.5: An off-topic question. Similarity was below the cutoff, so the model was never called.

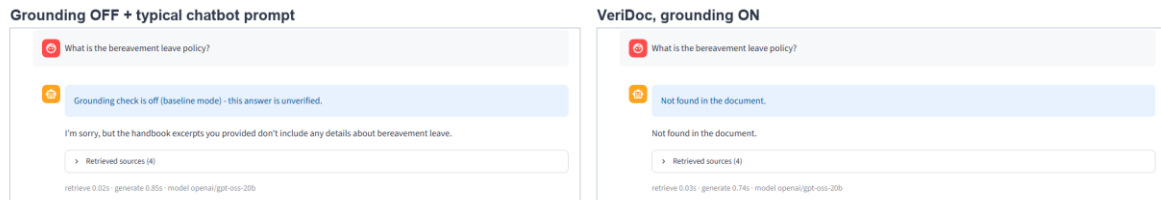


Figure 10.6: The same unanswerable question with grounding off and a typical prompt (left) and with VeriDoc (right). In this run the plain chatbot also refused, so it did not hallucinate; the difference is that the left answer is labelled unverified.

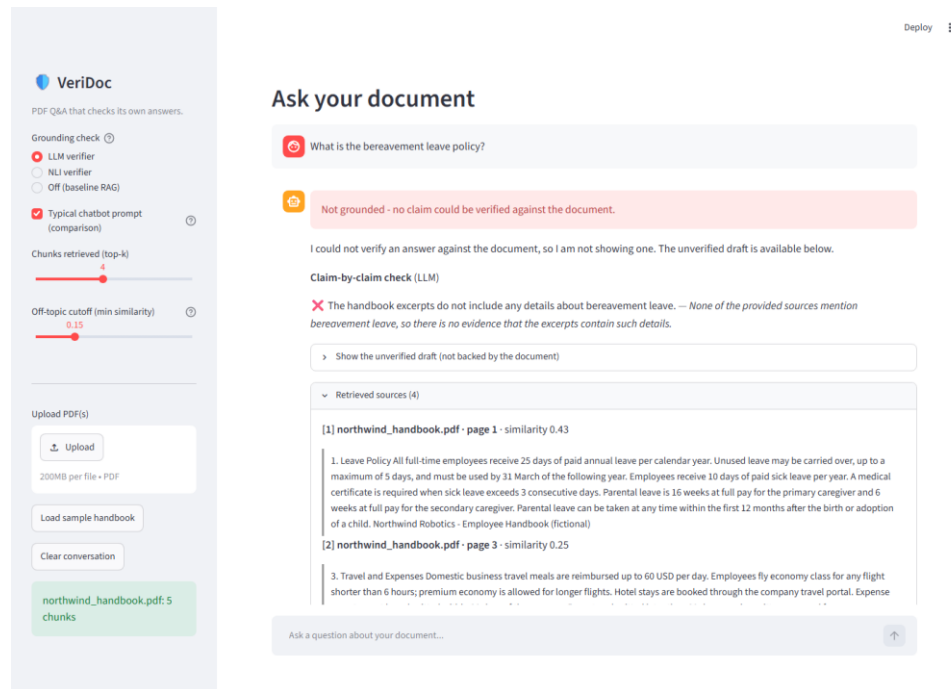


Figure 10.7: A limitation. With the typical prompt and the LLM verifier, a correct refusal is marked “Not grounded” because a negative statement has no quote to back it.

11. Challenges and Learnings

Problem I hit	How I solved it
The model's refusal NOT_FOUND was not recognised: the check compared text literally and the underscore broke it (caught by the unit tests).	Compare only the letters, case-insensitively, and added a test.
Some PDFs deliver one text block per visual line, so joining blocks with blank lines cut sentences in half.	Join blocks with a new paragraph only after a finished sentence or a short heading.
A bare list number such as "1." was treated as the end of a sentence.	Re-attach a lone number to the text that follows it.
The Groq model in the brief (Llama-3.3-70B) returned HTTP 404 – it has been retired.	Listed the models my key can use, tested JSON mode on the candidates, and switched the default to openai/gpt-oss-120b. It spends hidden reasoning tokens, so I set low reasoning effort.
The NLI model gave 0.002 for a supported claim when the evidence had two sentences.	Also score each single sentence; kept the best score. Verified in a regression test.
The model wrote narrow no-break spaces (U+202F) and 【 1 】 style citations. They broke sentence splitting, the NLI model and my scorer, which produced a misleading first result (NLI answered 42%).	Normalise model output right after generation; make the scorer accept number words and Unicode spaces. Re-ran everything.
The free tier of Groq allows 200,000 tokens per day per model. My evaluation runs used it up and one run crashed with HTTP 429.	Added an automatic fallback to another model when the daily cap is hit, recorded which model answered, and added a test. Ran the last experiment only on the development set.
My expectation was wrong: the baseline did not hallucinate.	Reported it as it is, added the typical-prompt experiment, and read the answers by hand instead of trusting the automatic metric.
A file was truncated when my edit script failed on Windows' default text encoding.	Restored the file from git and used UTF-8 explicitly. A reminder to commit often.
The GitHub login stored on the machine had expired, so pushes failed.	Committed locally and asked for a fresh sign-in.

What I learned. A plausible design idea should be tested against a baseline before believing it: on this setup the baseline was already good, so the honest value of VeriDoc is visibility (page numbers, quotes, per-claim marks) and a safety net, not a measured drop in hallucinations. Real model output is messier than test data, so invisible characters and formatting habits matter. And small NLI models are brittle on paraphrase.

12. Future Improvements

- **Treat refusals as valid.** Detect when an answer only says the document lacks the information and skip claim verification, which removes the false "Not grounded" cases seen in Figure 10.7.
- **A harder test.** Evaluate on a long real PDF (50+ pages) so retrieval is actually selective, with more questions and a weaker generator, to see whether verification reduces hallucination there.
- **Fine-tune the NLI verifier** on question-and-answer pairs from handbooks, or try a larger NLI model, to fix its trouble with numbers and paraphrase.
- **OCR support** for scanned PDFs, and **multi-turn memory** so follow-up questions work.
- **Cross-page chunking** for facts that continue across a page break, and saved sessions in a database such as Supabase.

13. Conclusion

VeriDoc is a working PDF chatbot that shows, for every answer, which claims the document supports, on which page, with which exact words. It refuses when the document has no answer and refuses without calling the model when the question is clearly off-topic. It has two verifiers that can be compared, an offline test suite, and a public interface.

Objective	Result	Met?
O1: answer at least 90% of answerable questions	Baseline and LLM verifier: 100% on both sets. NLI verifier: 92% (development), 80% (held-out)	Yes for baseline and LLM; No for NLI on held-out
O2: refuse at least 90% of unanswerable questions	100% on all configurations and both sets	Yes
O3: grounded precision of at least 95%	LLM and NLI verifiers: 100% on both sets	Yes
O4: verifier failure never shown as grounded	3 unit-test cases (bad JSON, garbage, crash) give unverified	Yes
O5: local verifier compared with LLM verifier	Done (Section 9); the local one is cheaper but less reliable	Yes
O6: at least 20 offline tests and a public app	25 tests pass. Live app: link [to be added before submission]	Tests: yes. App: see Section 14

The main lesson is that the evaluation was harder to get right than the system: fixing invisible output problems, keeping a held-out set, and reading answers by hand changed the conclusion from “baseline hallucinates, VeriDoc fixes it” to “the baseline is already careful here, and VeriDoc adds transparency, with its own weaknesses”.

14. References and Links

Project links

Source code	https://github.com/AnuruddhaPaul/VeriDoc
Live app	[to be added before submission]
Google Drive folder	[to be added before submission]

Libraries and services used

- PyMuPDF documentation: <https://pymupdf.readthedocs.io>
- Sentence-Transformers documentation: <https://www.sbert.net>
- ChromaDB documentation: <https://docs.trychroma.com>
- Groq API documentation: <https://console.groq.com/docs>
- Streamlit documentation: <https://docs.streamlit.io>
- Playwright for Python: <https://playwright.dev/python>

Papers and models

- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
- Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP.

- Wang, W. et al. (2020). MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. NeurIPS (basis of all-MiniLM-L6-v2).
- He, P., Gao, J. and Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. arXiv:2111.09543 (basis of cross-encoder/nli-deberta-v3-small).
- Williams, A. et al. (2018). A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference (MultiNLI). NAACL-HLT. The kind of data NLI cross-encoders are trained on.